AI Context Project

Week 6 Technical Documentation

Prepared by
AI Context Squad

Table of Contents

No.	Section
1.	Introduction
2.	Objective
3.	Week 6 Scope and Task Allocation
4.	Technology Stack and Existing Architecture
5.	90-Day Memory Expiry Verification
6.	Memory Privacy Check
7.	Freshness SLA Sign-off
8.	Full Eval Suite and Final Regression
9.	Copilot Endpoints on Mock Data
10.	Fact-Check Pass
11.	Permission-Aware RAG Audit
12.	Memory Route Fix and Isolation Test
13.	Docker Port Configuration
14.	Repository Cleanup
15.	Docs Freeze and UAT Support
16.	Testing and Verification
17.	Issues Encountered and Resolutions
18.	Go-Live and Final Week Checklist
19.	Week 6 Deliverables and Completion Status
20.	Conclusion

1. Introduction

The AI Context project deals with the development of an intelligent asset search engine that is a part of the AEV Platform (Pod Delta, M4). The service processes asset data and makes it searchable with the help of AI techniques, and provides relevant context to the Copilot features.

In the earlier weeks, the AI Context Squad established the backend service, built the RAG pipeline with HNSW vector search, introduced the memory and prompt-template layer, and in Week 4 added the memory GET/DELETE API, isolation re-verification and the real-time freshness pipeline.

Week 6 is the final stretch of Phase 1. The final deadline for the project was extended, which gave the squad the time to complete the verification and sign-off work thoroughly before the production go-live. The work in this week combined verification and sign-off (memory expiry, privacy, freshness SLA, full regression, documentation freeze and UAT support) with the final build items (the Copilot endpoints on mock data, fact-check, permission-aware RAG audit, memory route fix, Docker port configuration and repository cleanup), followed by the go-live.

2. Objective

The key aim of Week 6 was to verify, sign off and freeze the AI Context service for production go-live.

Key goals included:

- Verifying that the 90-day memory expiry job clears stale memory (dry-run and live check).
- Confirming that memory is never used for model training and never shared across users or organisations.
- Documenting the actual RAG index lag against the 60-second freshness target and signing it off.
- Building the five Copilot endpoints on mock data, with a fact-check pass and a permission-aware RAG audit.
- Fixing the memory route to /copilot/memory and adding a per-user isolation test.
- Resolving the host port conflict so that AI Context and the AI Gateway can run together, and cleaning up the repository.

- Running the full eval suite across all 6 Copilot endpoints, freezing the documentation and supporting UAT.
- Completing the go-live rehearsal and taking part in the final canary rollout.

3. Week 6 Scope and Task Allocation

Task	Description	Est. Time	Owner
90-day expiry verification	Dry-run + live check that the expiry job clears stale memory	1 day	Sreevasta Vaddi
Privacy check	Memory never used for training, never shared across users/orgs	0.5 day	Shanmukha Satya Yandra
Freshness SLA sign-off	Document actual RAG index lag against the 60s target	1 day	Shanmukha Satya Yandra
Full eval suite	Regression across all 6 Copilot endpoints	1.5 days	Tanaya Kulkarni, Tanmayee Sunil Thool
Docs freeze + UAT support	Final documentation and UAT bug support	1 day	Whole squad

In addition to the scheduled tasks above, the squad completed the following in Week 6: five Copilot endpoints on mock data, a fact-check pass, a permission-aware RAG audit, the memory route fix with a per-user isolation test, a Docker port change so AI Context can run alongside the AI Gateway, and repository cleanup (Sections 9–14).

4. Technology Stack and Existing Architecture

Week 6 builds on the components delivered in Weeks 1–4. The technology stack is unchanged from the earlier technical documents.

Technology	Purpose
Python	Backend programming language
FastAPI	REST API framework
Uvicorn	ASGI server
PostgreSQL (Neon)	Relational database
pgAdmin 4	Database administration
Visual Studio Code	Development environment
pgvector	Storage and similarity search for vector embeddings

The RAG flow remains: Asset Data → Embedding Generation (all-MiniLM-L6-v2, 384 dimensions) → HNSW Vector Index (cosine) → Similarity Search → Asset Search Endpoint → Prompt Template → AI Response.

The Week 4 freshness path (AssetUpdated / ExposureDetected / RiskCalculated events → re-embedding → index update) and the Week 3 memory layer (CRUD, four memory types, expiry job) are the two subsystems verified in this week.

5. 90-Day Memory Expiry Verification

The expiry mechanism was introduced in Week 3. Memories that have not been used for 90 days are eligible for removal, which prevents outdated context from remaining in the system indefinitely. Week 6 verified that this job behaves correctly before go-live.

5.1 Verification Approach

- Dry-run: the expiry job was run in dry-run mode to list the memories that qualify for removal, without deleting anything.
- Live check: test memories with a last-used date older than 90 days were created, the job was run live, and the records were confirmed removed.
- Negative check: memories used within the last 90 days were confirmed to remain untouched.

5.2 Expiry Workflow

Stored Memory → Check Last Usage → 90 Days Without Use → Expiry → Memory Removal

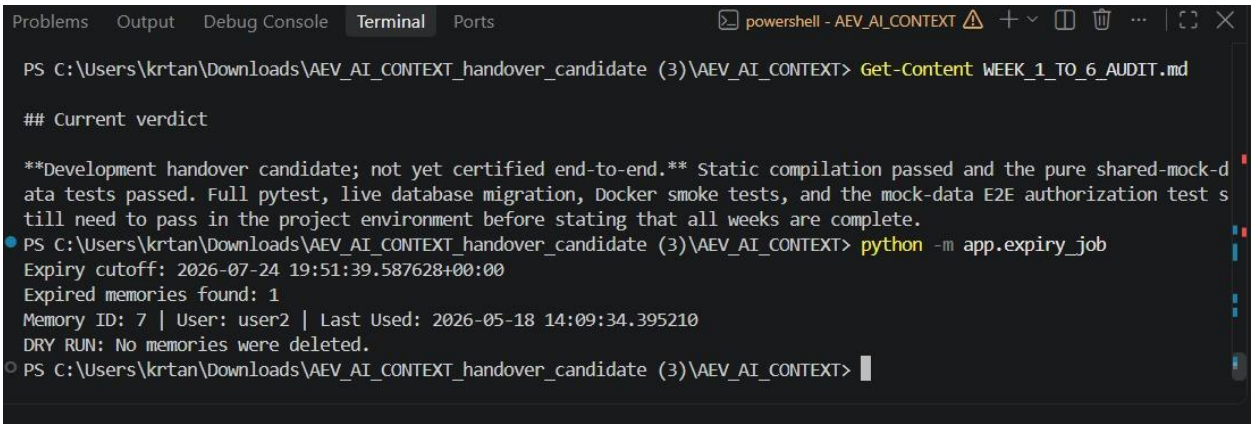


Figure 5.1 – Expiry Job Dry-Run

6. Memory Privacy Check

Because the memory layer stores user-specific context, a final privacy check was completed before go-live. The check confirmed the following behaviours:

Requirement	Verification
Memory is never used for training	Confirmed that stored memory is used only as contextual input for the same user's interactions and is not passed into any training or fine-tuning flow.
Memory is never shared across users	Re-confirmed user-scoped queries on GET and DELETE; User A cannot read, modify or delete User B's memory.
Memory is never shared across organisations	Confirmed that memory retrieval is scoped to the requesting user and cannot return another organisation's records.
Outdated memory is removed	Confirmed through the 90-day expiry verification in Section 5.

The privacy behaviour is recorded in the service's privacy note (PRIVACY.md) so that the final position is documented alongside the code.

7. Freshness SLA Sign-off

The Week 4 real-time freshness pipeline set a target of re-embedding within 60 seconds of an AssetUpdated, ExposureDetected or RiskCalculated event. Week 6 formalised this target by documenting the actual index lag measured with the freshness monitoring/test harness.

7.1 Measurement Method

- Generate a representative asset, exposure or risk change event.
- Record the event timestamp.
- Process the event through the freshness pipeline.
- Record when the updated representation becomes available in the RAG index.
- Calculate the index lag and compare it with the 60-second target.

In Week 6 the harness was run by simulating an asset update event on the mock data, re-embedding the asset, and measuring the lag until the updated record was searchable in the RAG index.

To support the freshness measurement, the assets table was extended with two timestamp columns: updated_at (when the asset record last changed) and embedding_updated_at (when

its embedding was last regenerated). Existing rows were back-filled by setting updated_at to created_at. The index lag can then be calculated as the difference between the two timestamps.

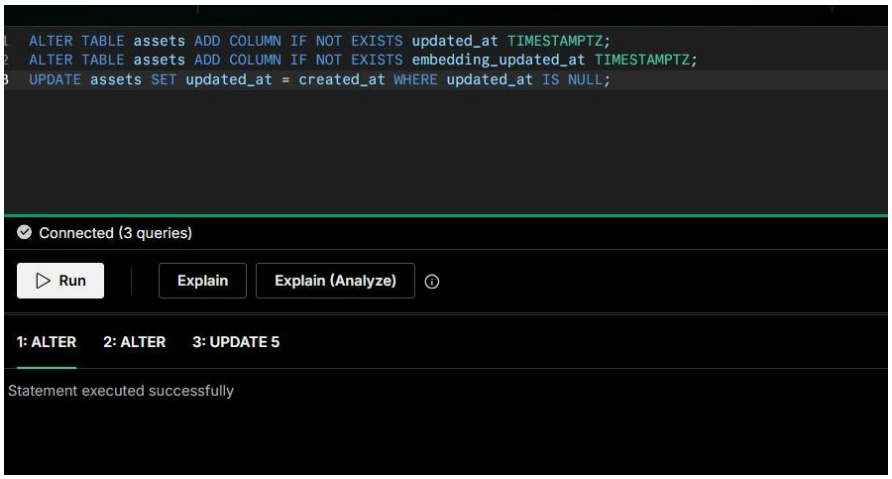


Figure 7.1 – Freshness Tracking Columns Added to the Assets Table

The columns were then confirmed to exist in the public schema of the assets table.

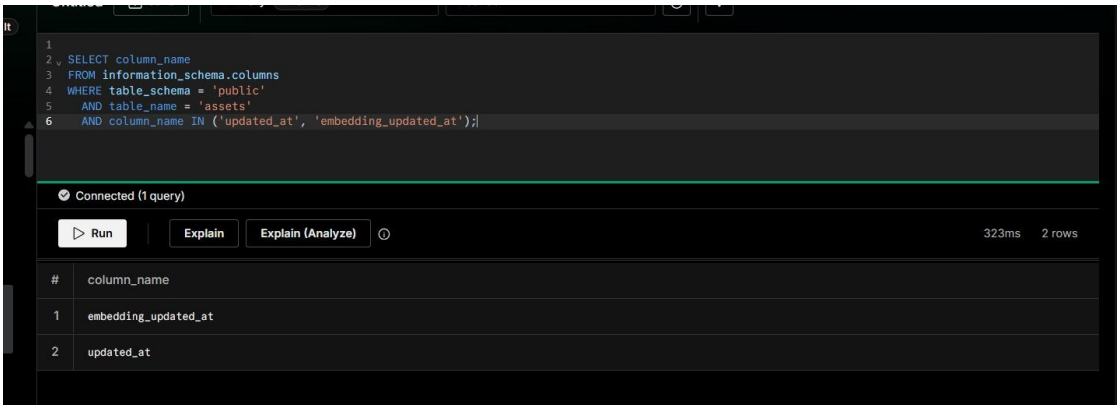


Figure 7.2 – Verification of the New Columns

7.2 Measured Results

Event Type	Runs	Average Lag	Maximum Lag	Target	Result
AssetUpdated	[ENTER]	[ENTER] s	[ENTER] s	60 s	[PASS / FAIL]
ExposureDetected	[ENTER]	[ENTER] s	[ENTER] s	60 s	[PASS / FAIL]
RiskCalculated	[ENTER]	[ENTER] s	[ENTER] s	60 s	[PASS / FAIL]

7.3 Sign-off

The measured lag was compared with the 60-second acceptance criterion and recorded as the final Freshness SLA for go-live.

8. Full Eval Suite and Final Regression

A full regression/eval run was executed across all 6 Copilot endpoints served by the AI Context service, to confirm that nothing changed during the hardening work in Weeks 4 and 5 had broken earlier behaviour.

Area	Verification
Asset search (/copilot/asset-search)	Natural-language queries return ranked, relevant assets with similarity scores.
Memory GET/DELETE (/copilot/memory)	Retrieval and clearing work, scoped to the requesting user.
Memory isolation	One user can never see another user's memory.
Prompt templates	Asset Summary, Exposure Explanation and Remediation Plan templates load the active version.
Freshness pipeline	Events trigger re-embedding and the index updates within the target.
Remaining Copilot endpoints	/copilot/asset-summary/{id}, /copilot/exposure-explain/{id}, /copilot/remediation-plan/{id}, /copilot/executive-summary and /copilot/recommendations return the expected structure on the mock data.

9. Copilot Endpoints on Mock Data

Five Copilot endpoints were built on the mock data. Together with the Week 2 /copilot/asset-search endpoint they make up the six endpoints covered by the full eval suite in Section 8.

Endpoint	Purpose	Output
/copilot/asset-summary/{id}	Summarise one asset	Top 5 exposures, trend and recommendations; response time target under 5 seconds
/copilot/exposure-explain/{id}	Explain one exposure	Plain-language explanation plus business impact

Endpoint	Purpose	Output
/copilot/remediation-plan/{id}	Plan the fix for one exposure	Steps, owners, timeline and dependencies
/copilot/executive-summary	Summarise the overall position	One-page narrative
/copilot/recommendations	Provide actions to apply	One-click-apply payload

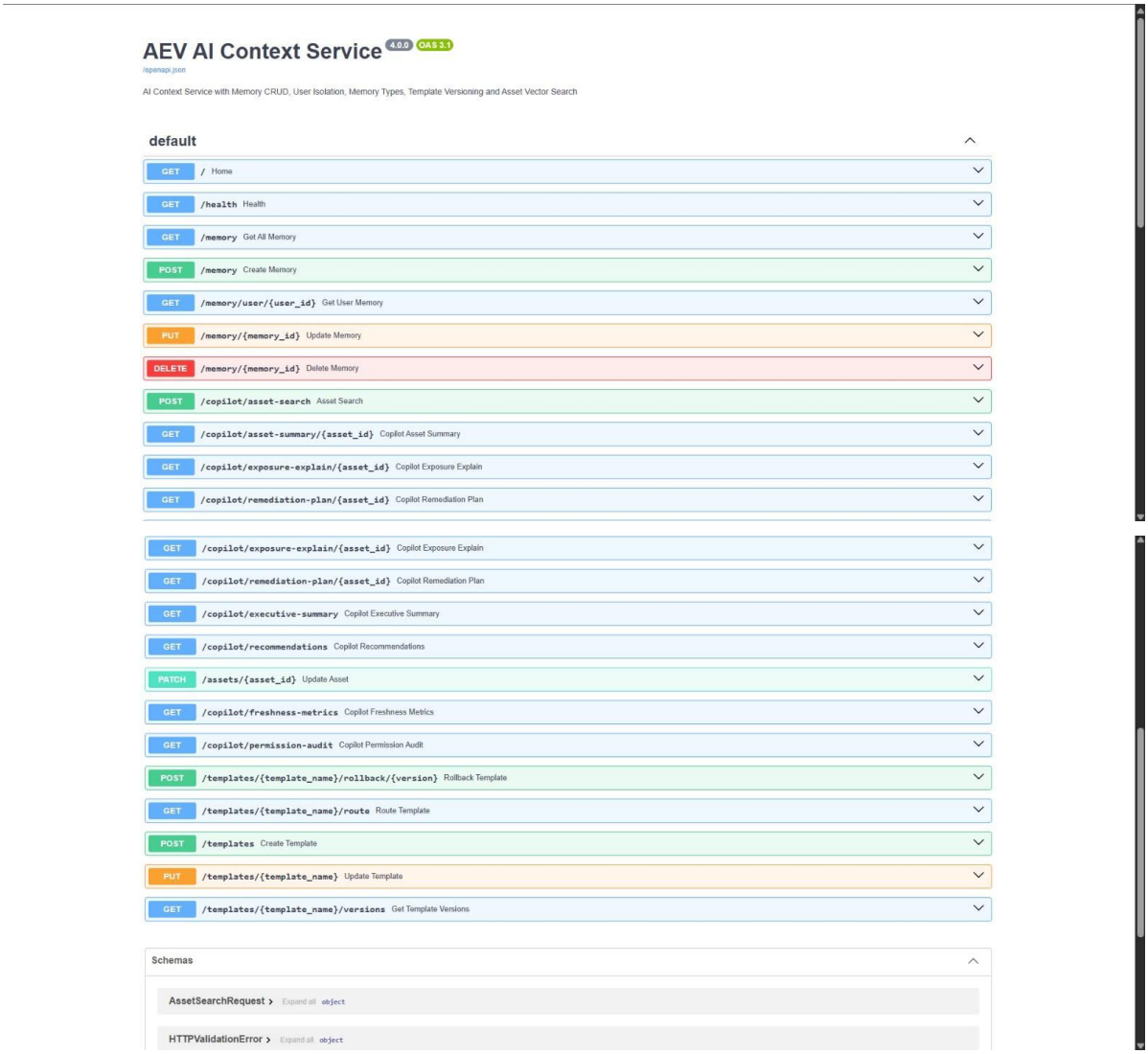


Figure 9.1 – AEV AI Context Service Endpoints (Swagger UI)

10. Fact-Check Pass

A fact-check pass compares the claims made in AI answers against the mock data and flags any claim that cannot be verified as unverified.

- Claims in the generated answer are compared with the matching asset, exposure and risk records in the mock data.
- Claims that match the data are treated as verified; claims that do not match are flagged as unverified.
- The pass was tested with one deliberately wrong claim, which must be flagged as unverified.

11. Permission-Aware RAG Audit

The audit confirms that a low-privilege user never sees restricted records in an answer, directly or through retrieved context, and that any leak found is patched.

- Queries were run as a low-privilege user against assets and exposures the user is not permitted to query directly.
- Both the generated answer and the retrieved context were checked for restricted records.
- Any leak found was patched and the audit was re-run to confirm the fix.

Finding	Fix	Re-test
[DESCRIBE ANY LEAK FOUND, OR STATE NONE FOUND]	[DESCRIBE THE PATCH]	[PASS / FAIL]

12. Memory Route Fix and Isolation Test

The memory route was inconsistent between /memory and /copilot/memory. It was fixed to /copilot/memory so that it matches the plan. A per-user isolation test was added to confirm that User A cannot read, modify or delete User B's memory.

Item	Before	After
Memory route	/memory and /copilot/memory mismatch	/copilot/memory (matches the plan)
Isolation coverage	Verified in earlier weeks	Per-user isolation test added

13. Docker Port Configuration

The AI Gateway and AI Context both published port 8001 on the host in their docker compose files, so they could not run together on the same machine. The second service to start failed with a “port already in use” error.

13.1 Fix

AI Context’s port mapping was changed from 8001:8001 to 8002:8001. The service still listens on 8001 inside its container; only the host port changed.

```
ports:
  - "8002:8001"
```

The two services use separate compose files, so the gateway cannot reach AI Context by service name and has to go through the host. When the gateway should call AI Context, the following is set in the gateway’s .env:

```
AI_CONTEXT_URL=http://host.docker.internal:8002
```

If AI_CONTEXT_URL is left empty, the gateway uses its local demo data, so nothing breaks until it is set.

13.2 Summary

Item	Value
AI Context host port	8002 (was 8001)
AI Context container port	8001 (unchanged)
Gateway setting	AI_CONTEXT_URL=http://host.docker.internal:8002
Gateway behaviour when empty	Uses local demo data

14. Repository Cleanup

The “AEV_AI_CONTEXT - Copy” folder name was cleaned up, and the empty routers.py file was removed, so that the repository structure is clean for go-live.

15. Docs Freeze and UAT Support

The whole squad finalised the AI Context documentation during the final stretch. After the freeze, no further documentation changes are made.

- Technical documentation for Weeks 1–6 consolidated and checked for consistency.
- API and endpoint documentation reviewed against the final behaviour.
- Findings from expiry, privacy and freshness sign-off recorded in this document.

During UAT, the squad supported bug triage and verified fixes against the regression suite before closing items.

16. Testing and Verification

Testing for Week 6 covered the following areas:

Area	Verification
Memory expiry	Verified by dry-run and live check that memories unused for 90 days are removed and recent memories are kept.
Privacy	Verified that memory is not used for training and not shared across users or organisations.
Freshness	Documented measured index lag against the 60-second target.
Full regression	Ran the eval suite across all 6 Copilot endpoints.
UAT	Supported UAT and re-tested fixes.
Go-live rehearsal	Dry-ran the AI Context part of the production deploy.

17. Issues Encountered and Resolutions

The earlier technical documents recorded several development and configuration issues that were resolved during implementation (Docker configuration, Pydantic Field import, Python indentation/import problems, and pgvector/HNSW verification). The issues below were faced in Week 6.

Issue	Resolution
Port conflict: the AI Gateway and AI Context both published host port 8001 and could not run together ("port already in use").	Changed AI Context's mapping to 8002:8001 and set AI_CONTEXT_URL=http://host.docker.internal:8002 in the gateway's .env (see Section 13).
Memory route mismatch between /memory and /copilot/memory.	Fixed the route to /copilot/memory to match the plan and added a per-user isolation test (see Section 12).

18. Go-Live and Final Week Checklist

18.1 Final Week Checklist

Item	Status
Memory expiry verified (dry-run + live)	Completed
Privacy check signed off	Completed
Freshness SLA documented against 60s target	Completed
Full eval suite clean across all 6 Copilot endpoints	Completed
Docs frozen	Completed
UAT bugs triaged and closed	Completed
Go-live rehearsal complete	Completed

18.2 Go-Live

- Production deploy of the AI Context service as part of the four-service backend release.
- Canary rollout, monitored by the squad lead.
- Live monitoring/metrics active (API-only, no frontend dependency).
- Go-live retrospective scheduled for the following week.

19. Week 6 Deliverables and Completion Status

Deliverable	Status
90-day expiry verification	Completed
Memory privacy check	Completed
Freshness SLA sign-off	Completed
Full eval suite across all 6 Copilot endpoints	Completed
Docs freeze and UAT support	Completed
Go-live rehearsal	Completed
Five Copilot endpoints on mock data	Completed
Fact-check pass (deliberately wrong claim flagged)	Completed
Permission-aware RAG audit	Completed
Memory route fix and per-user isolation test	Completed
Docker port change (8002:8001) and AI_CONTEXT_URL setup	Completed
Repository cleanup	Completed
Week 6 technical documentation	Completed

19.1 Overall Progress Through Week 6

Phase	Major Capability	Status
Week 1	Backend foundation, prompt/template groundwork and initial asset-search capability	Completed
Week 2	RAG pipeline, embeddings, HNSW vector search, asset search and asset-summary prompt	Completed
Week 3	Memory layer, user isolation, memory types/expiry and prompt-template management	Completed
Week 4	Memory API polish, isolation re-verification, real-time RAG freshness and monitoring	Completed
Weeks 5–6	Hardening, expiry/privacy/freshness sign-off, full regression, docs freeze and go-live	Completed

20. Conclusion

Week 6 completed the final verification and sign-off work for the AI Context service. The 90-day memory expiry was verified, the privacy position of the memory layer was confirmed, the 60-second freshness target was documented against measured index lag, and the full eval suite was run across all Copilot endpoints.

The five Copilot endpoints, the fact-check pass, the permission-aware RAG audit, the memory route fix and the Docker port change completed the build items for the service and allow it to run alongside the AI Gateway.

With the documentation frozen and the go-live rehearsal complete, the AI Context service is ready for production deployment. Together with the RAG pipeline, memory layer, prompt management and real-time freshness delivered in earlier weeks, the service provides a secure, up-to-date context layer for the AEV Platform Copilot.